Understanding the RAG Pipeline

1. Introduction

RAG (Retrieval-Augmented Generation) lets an LLM answer from **our own data**. The LLM never saw our manuals, contracts or policies, so before it answers we find the few pieces of text that matter and give them to it together with the question.

Why RAG? An LLM alone knows only its training data. It does not know our documents or anything newer, and when it does not know, it may invent an answer. RAG grounds every answer in real text, so answers are **more accurate**, **up to date** (we just update the documents, no retraining) and **traceable**, because they can cite their source. It is also much cheaper and faster than fine-tuning a model on our data.

A RAG system works in two phases. **Indexing** prepares the documents once, before any user asks anything. **Querying** runs every time a user asks a question. Both phases meet at the vector database.

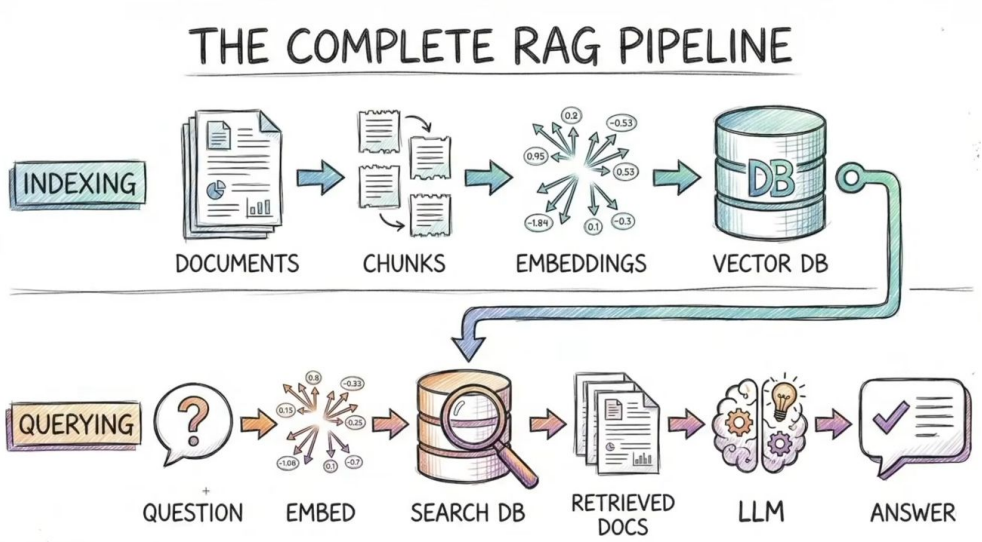


Figure 1: The complete RAG pipeline. Top row: indexing (prepare the data). Bottom row: querying (answer a question).

2. Phase 1: Indexing

Indexing has four steps: load, chunk, embed, store. Each step has one job.

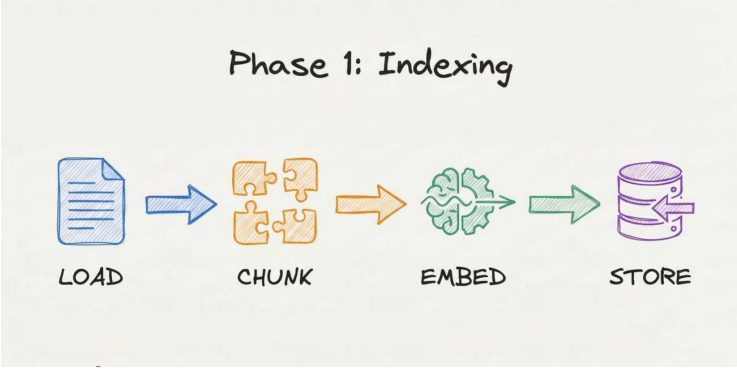


Figure 2: The four indexing steps.

- **Load:** read files (PDF, Word, web pages) into Documents with text and metadata.
- **Chunk:** split each Document into small pieces. *Why:* a small, focused piece matches a question much better than a whole 50-page manual, and models have input limits.
- **Embed:** turn each chunk into a vector (a list of numbers) that captures its meaning. *Why:* we can then search by meaning, not exact words: "late check-out" can find "departure after 12:00".
- **Store:** save vectors, chunk text and metadata in a vector database. *Why:* it finds the closest vectors among millions in milliseconds.

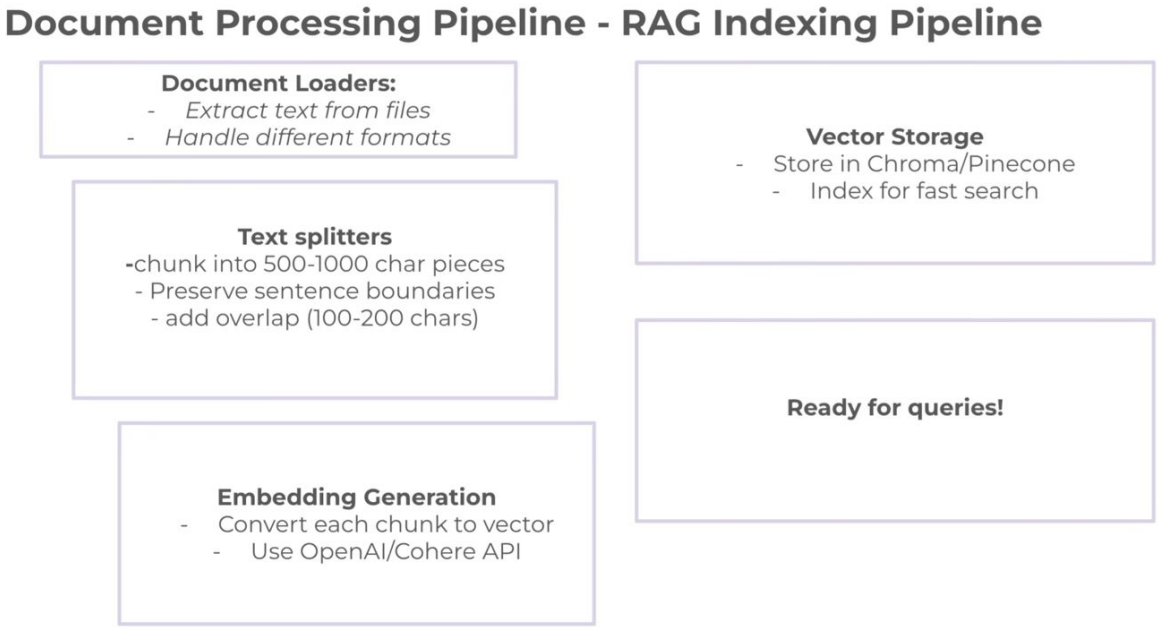


Figure 3: Summary of what each indexing component does.

> **Careful:** Figure 3 talks about **characters** (500-1000 chars), Figure 8 about **tokens** (200-1000 tokens). They are not the same: a token is usually more than one character, often a word or part of a word.

3. Chunking: Why It Matters

Chunking is not preprocessing, it is **architecture**. Chunking decisions affect the entire pipeline: even with the best model and the best input quality, bad chunking gives the worst results.

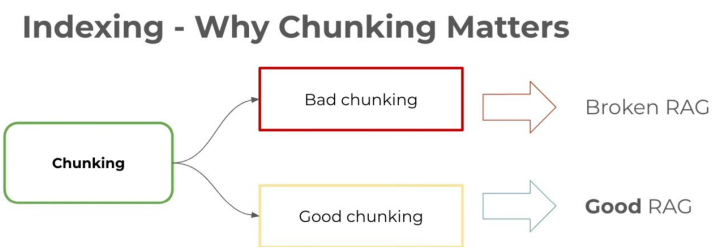


Figure 4: The same pipeline with bad or good chunking gives a broken or a good RAG.

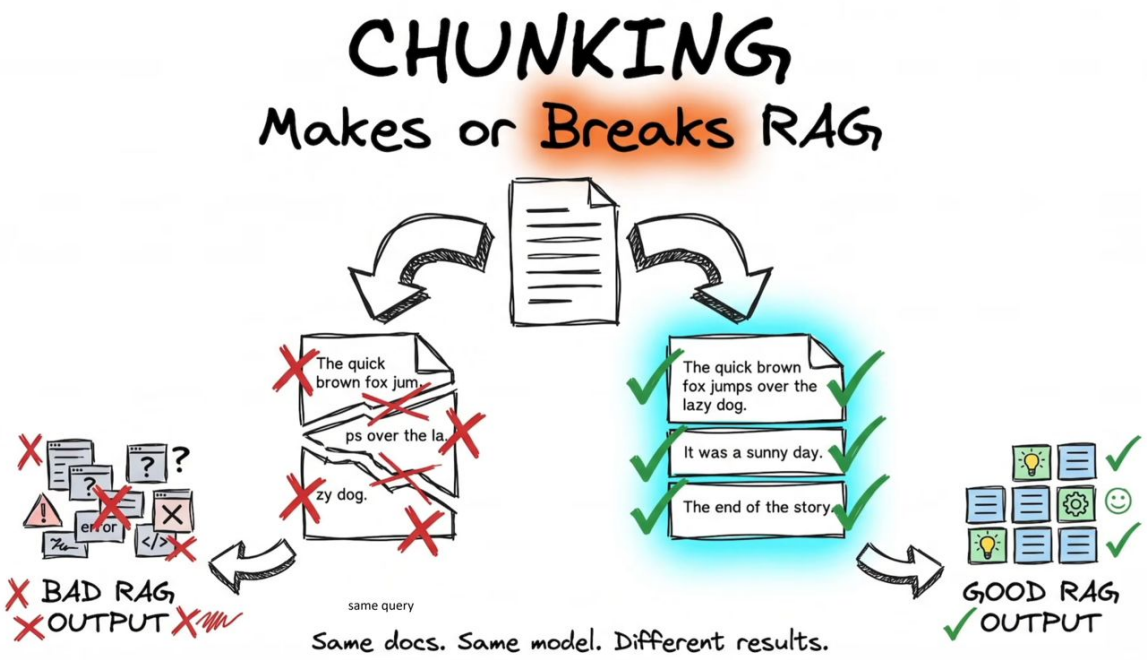


Figure 5: Same documents, same model, same query. Only the chunking changes, and so does the answer.

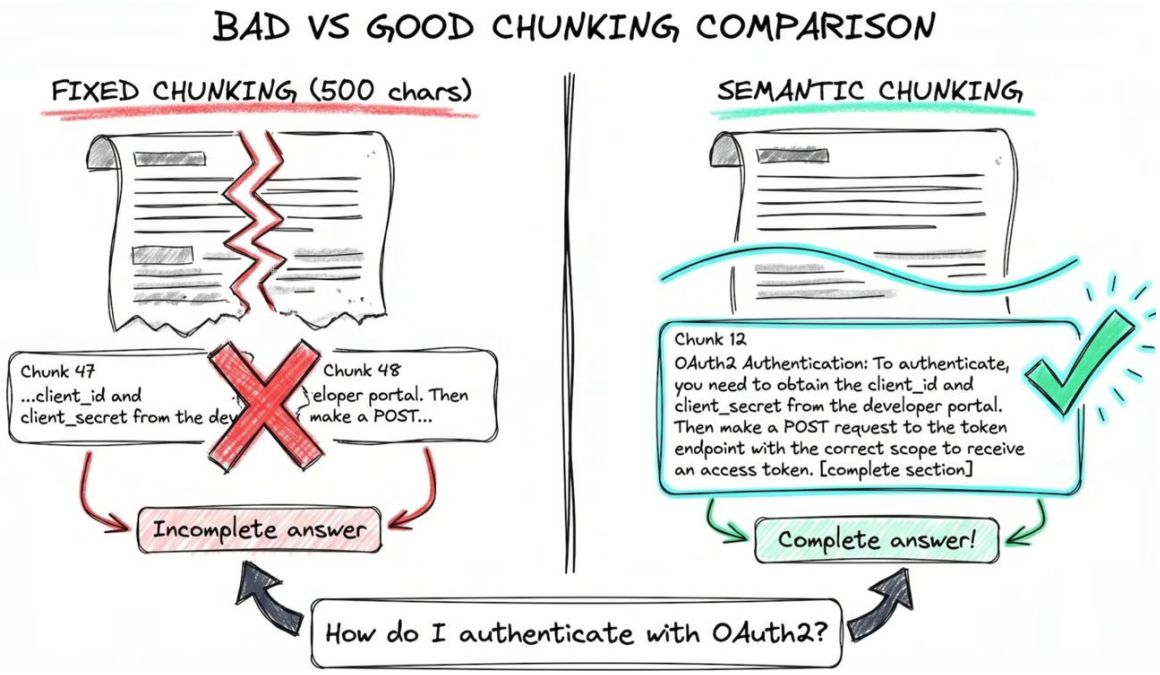


Figure 6: Fixed chunking splits one instruction in two; semantic chunking keeps it whole.

In Figure 6 the user asks how to authenticate with OAuth2. Fixed-size chunking cuts the instructions in the middle, so chunk 47 has half the steps and chunk 48 the other half. The retriever brings back one of them and the answer is incomplete. Semantic chunking keeps the whole section in one chunk.

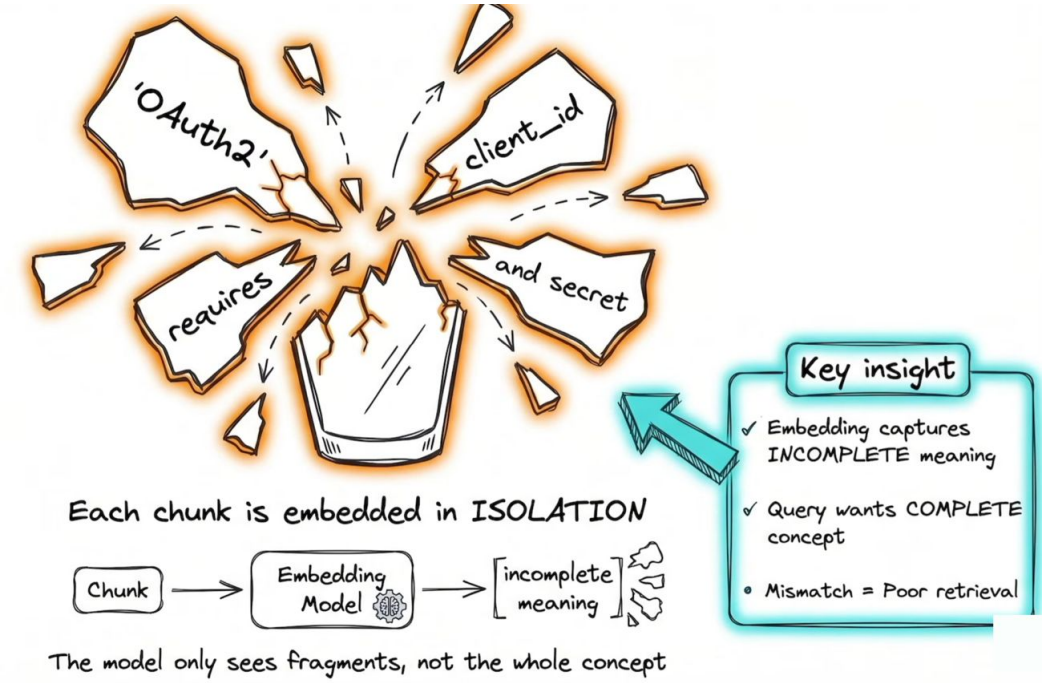


Figure 7: Each chunk is embedded in isolation, so a broken chunk gets a vector with incomplete meaning.

Figure 7 shows the root cause: every chunk is embedded **alone**. The embedding model only sees the fragment, so the vector describes half an idea, while the question asks for the full idea. This mismatch means poor retrieval.

3.1 The Garbage Chain: An Example

Bad chunking -> garbage embeddings -> garbage retrievals -> bad answer.

A hotel policy document says:

```
"Free cancellation up to 48 hours before arrival. After that,
 the first night is charged. Non-refundable rates cannot be
 cancelled for free."
```

Bad chunking puts the first sentence in one chunk and the rest in other chunks. A guest asks: *"I booked a non-refundable rate for next week. Can I cancel for free?"* The retriever returns the chunk that matches "cancel for free" best, the first sentence. The LLM answers:

```
"Yes, you can cancel for free up to 48 hours before arrival."
```

This is **wrong**, and it costs the hotel money. The LLM did nothing wrong: it answered correctly from the text it got. The rule that mattered was in a chunk that was never retrieved.

4. Four Chunking Variables

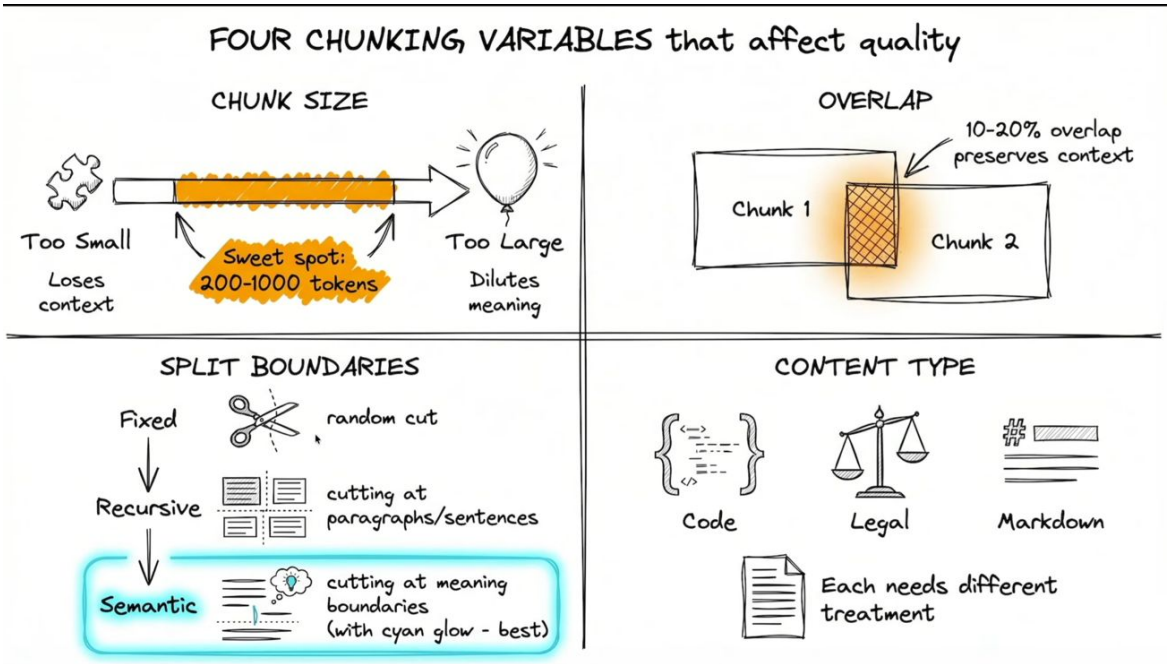


Figure 8: The four choices that decide chunk quality.

- **Chunk size:** too small loses context, too large mixes topics and dilutes meaning. Sweet spot: 200-1000 tokens.
- **Overlap:** repeat 10-20% of the end of one chunk at the start of the next, so a sentence near a border is not lost.
- **Split boundaries:** where we cut: at fixed points, at paragraphs and sentences, or at changes of meaning.
- **Content type:** code, legal text and Markdown each need their own splitter.

5. Chunking Strategies

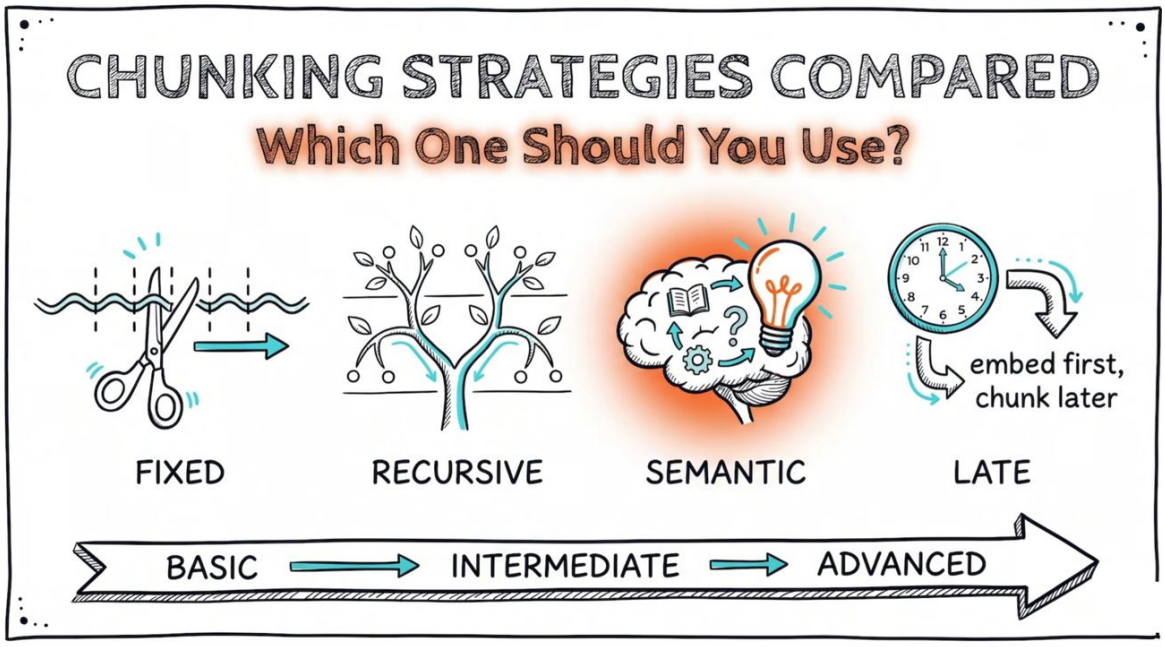


Figure 9: Four strategies, from basic to advanced.

5.1 Fixed-size Chunking

Cut every N characters, no matter what is there. Simple, fast and predictable, but it cuts words and sentences in half. Verdict: only for quick prototypes.

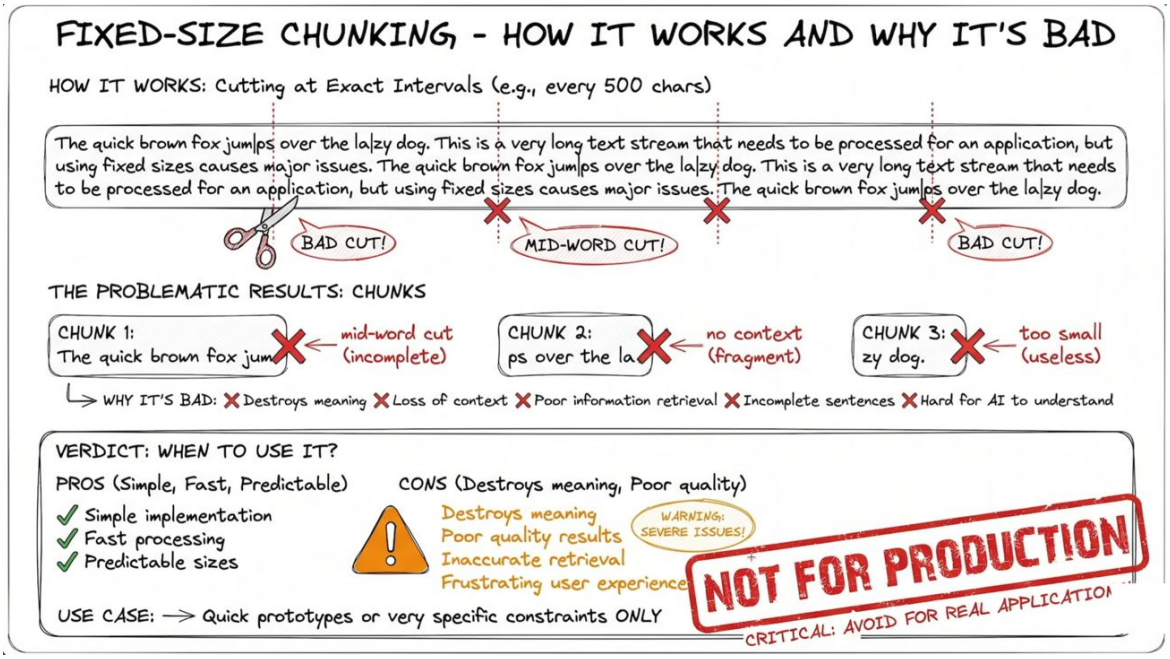


Figure 10: Fixed-size chunking cuts mid-word and leaves fragments without context.

5.2 Recursive Chunking

Try to split at paragraphs (\n\n) first. If a piece is still too big, split it at lines, then sentences, then clauses, then words. Characters are the last resort. Chunks follow the natural structure of the text. It is the recommended default.

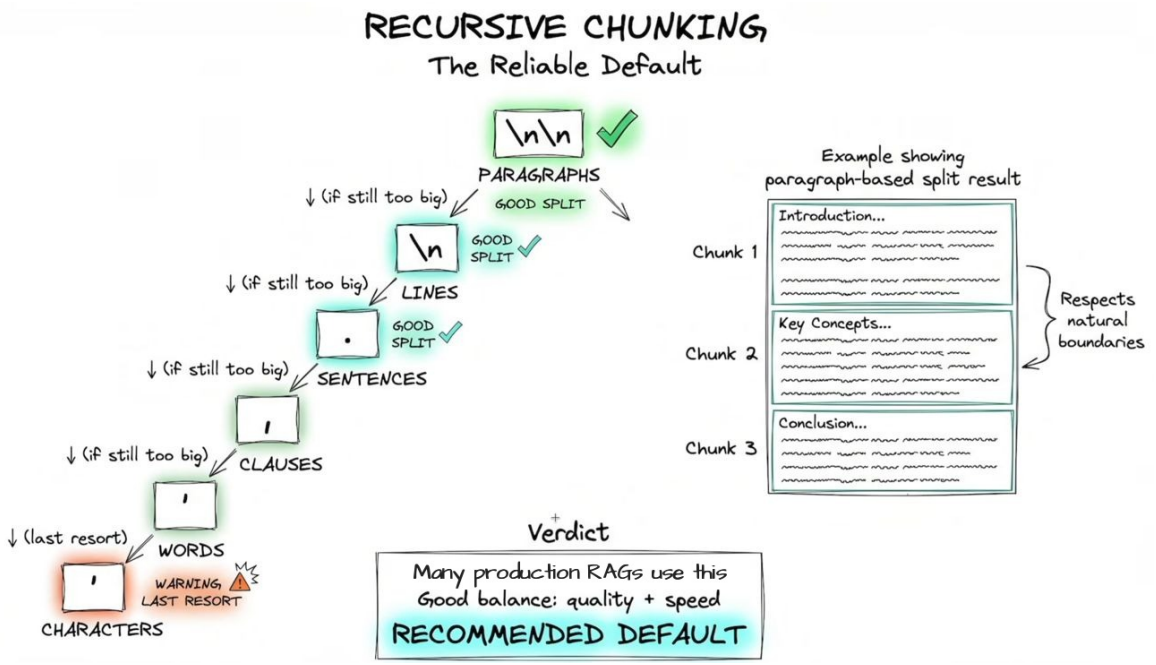


Figure 11: Recursive chunking goes down to smaller separators only when a piece is still too big.

5.3 Semantic Chunking

Embed every sentence, compare each sentence with the next one, and cut where the similarity drops: that is where the topic changes. The cost is speed and money, since every sentence must be embedded during indexing.

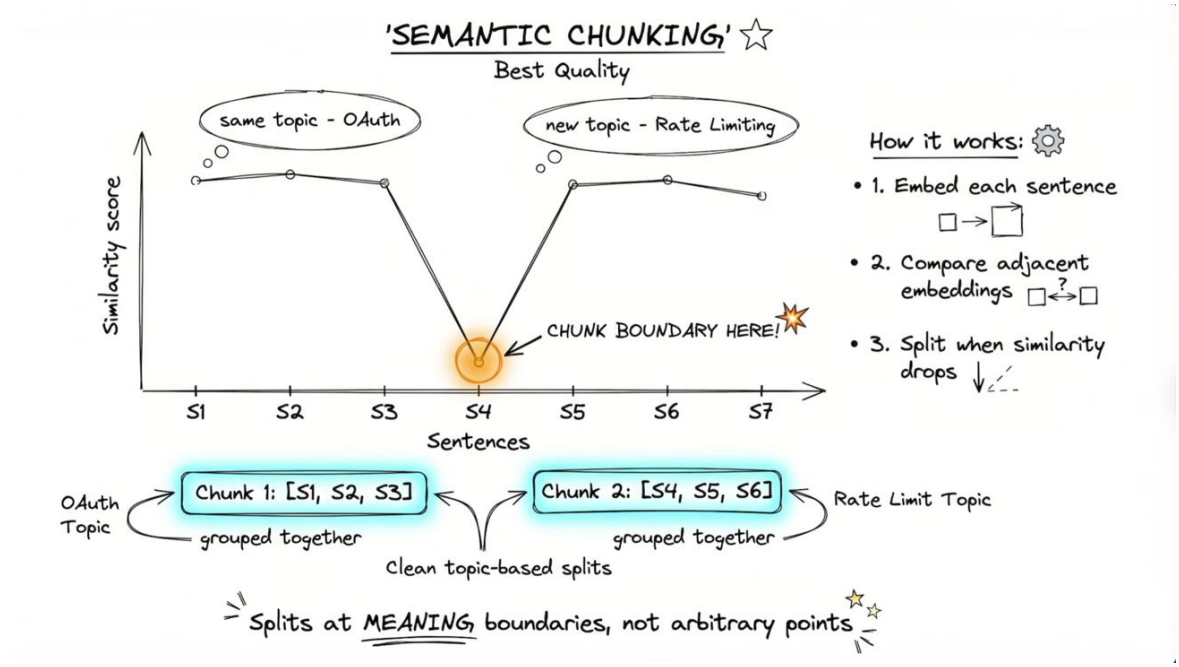


Figure 12: Similarity between neighbour sentences drops at S4, where the topic changes from OAuth to rate limiting.

Use semantic chunking when quality matters more than speed.

5.4 Late Chunking

Traditional chunking cuts first and embeds each chunk alone, so chunk 5 knows nothing about chunks 1-4. Late chunking embeds the **whole document first** with a long-context model, then cuts and averages (pools) the token vectors of each chunk. Each chunk vector now carries context from the rest of the document, e.g. it knows that "it" means the hotel named three paragraphs earlier. It needs an embedding model that supports it.

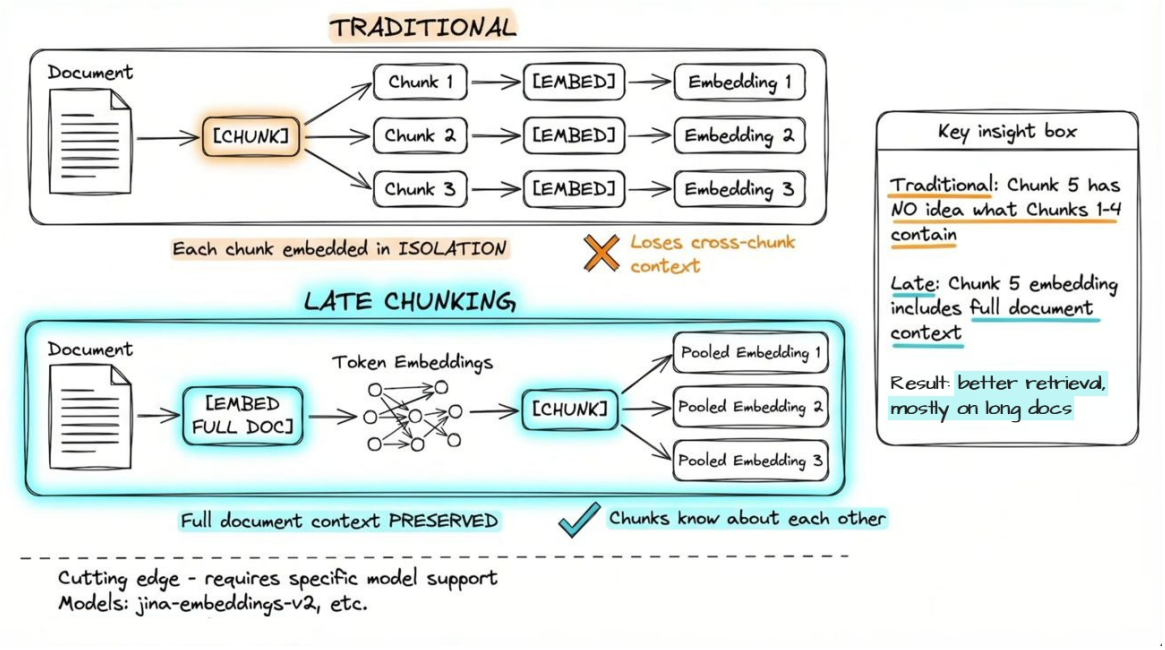


Figure 13: Traditional: chunk, then embed. Late: embed the full document, then chunk.

5.5 Which One to Use?

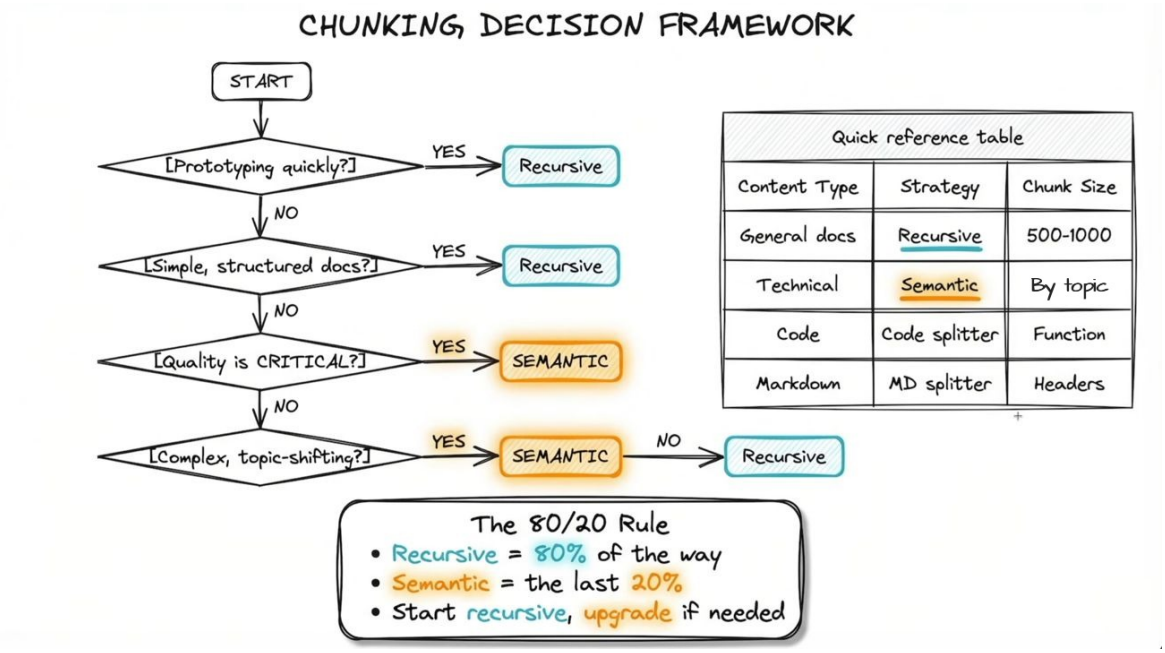


Figure 14: Decision framework: start with recursive, upgrade only if needed.

6. Embeddings

A chat model takes text and returns text. An embedding model takes text and returns a vector. Texts with similar meaning get vectors that are close to each other, and retrieval simply means "find the vectors closest to the question's vector" (usually with cosine similarity).

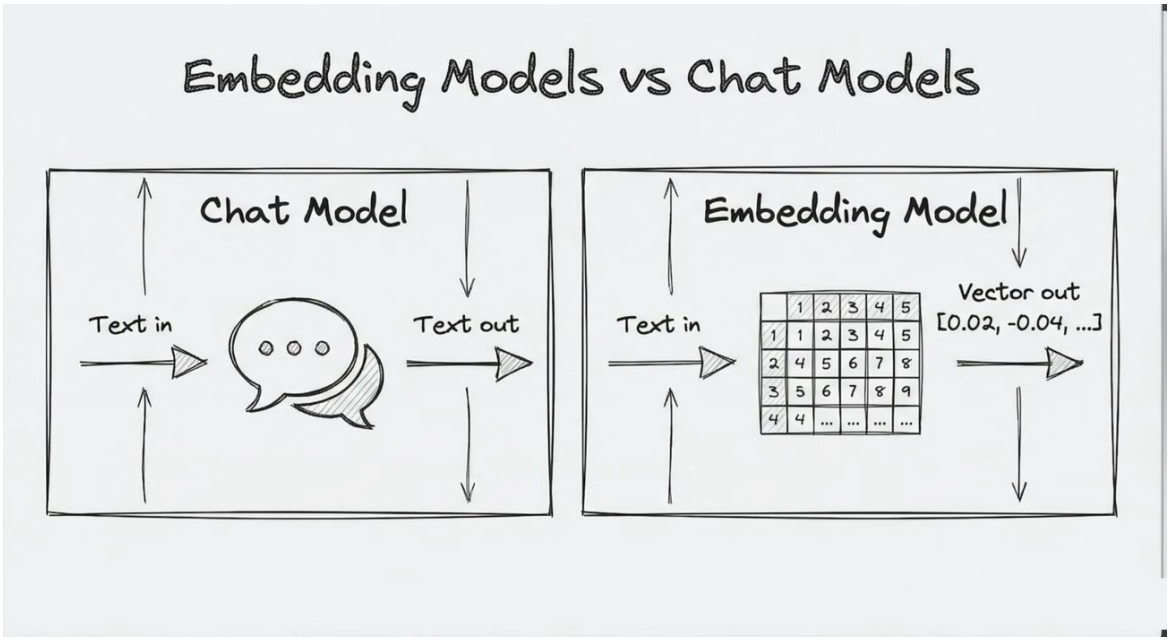


Figure 15: Chat model: text in, text out. Embedding model: text in, vector out.

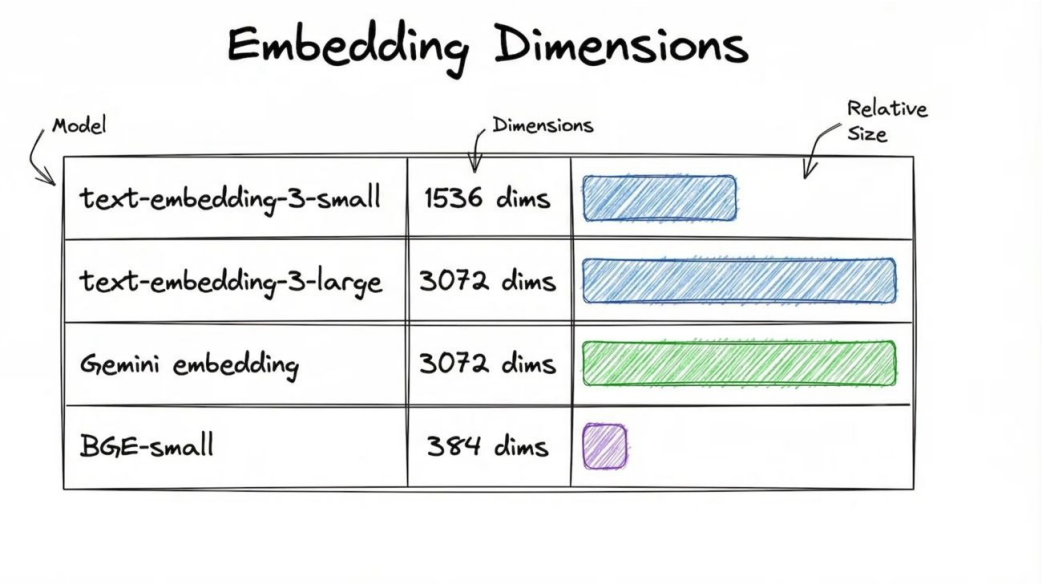


Figure 16: Dimensions (vector length) of some common embedding models.

More dimensions means more features of meaning can be captured, but more storage and slower search. Fewer dimensions means faster and cheaper.

Important: use the same embedding model in both phases. Vectors from two different models live in different spaces, like coordinates from two different maps. Comparing them gives an error (different lengths) or, worse, silent nonsense (same length).

7. Vector Storage

The vector database keeps each chunk's vector, text and metadata, and builds an index for fast approximate search. **Chroma** runs locally and is free, good for prototypes. **Pinecone** is a managed cloud service. Other common options: pgvector (inside PostgreSQL), Qdrant, FAISS.

8. Phase 2: Querying

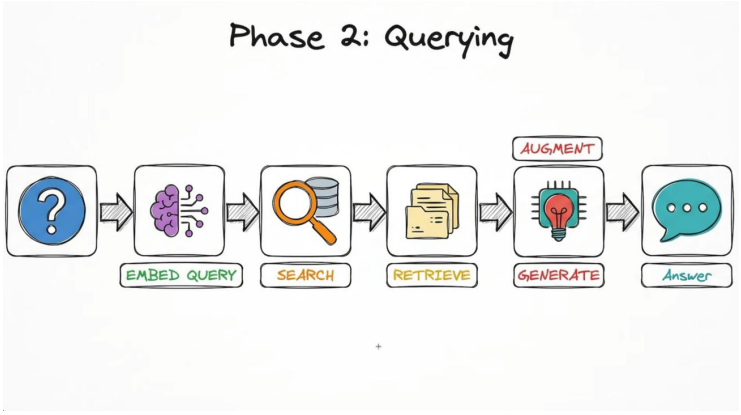


Figure 17: The querying steps, run for every user question.

- **Embed query:** turn the question into a vector, with the same model used for indexing.
- **Search:** find the top-k closest chunk vectors (e.g. k = 4).
- **Retrieve:** fetch the text and metadata of those chunks.
- **Augment:** put the chunks into the prompt, next to the question.
- **Generate:** the LLM writes the answer and can cite sources from the metadata.

9. Three Rules for Production RAG

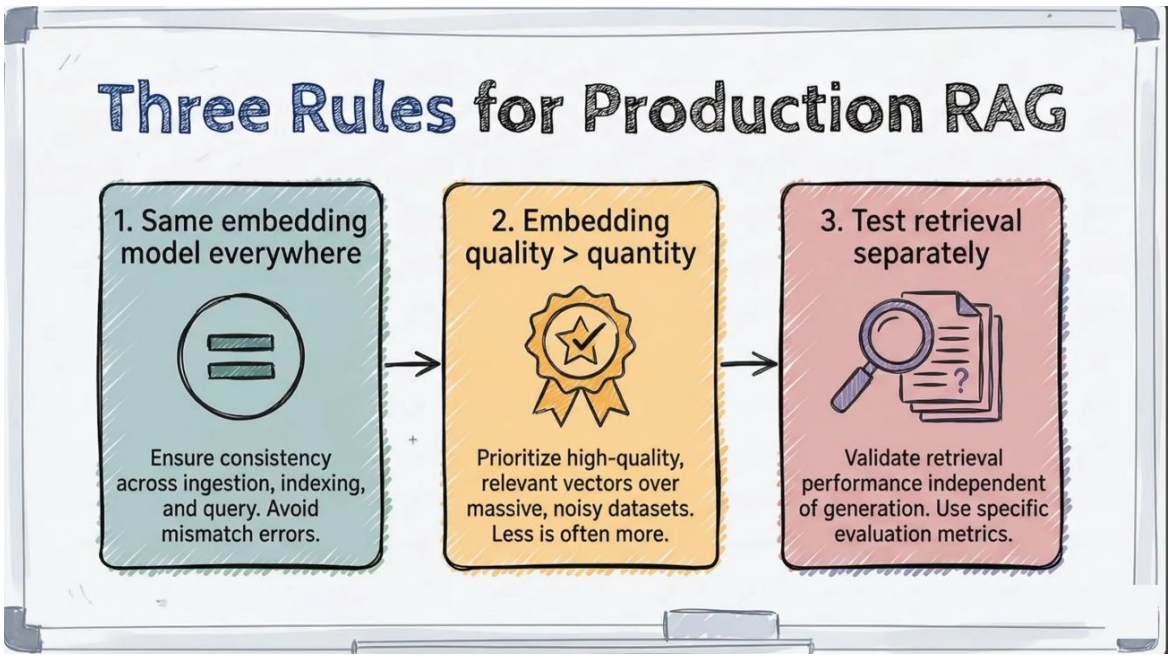


Figure 18: Three rules for a RAG system that works in production.

- **Same embedding model and version everywhere.** If you change the model, re-embed all documents. Store the model name in the metadata so you can check.
- **Embedding quality over quantity.** Better embeddings on fewer, clean, relevant documents beat worse embeddings on more documents. Noisy and duplicate chunks push the good ones out of the top-k.
- **Test retrieval separately.** Write 20-30 real questions, mark which chunk holds each answer, and check if that chunk appears in the top-k results (hit rate / recall@k). If retrieval fails, no prompt and no bigger LLM can fix the answer.

Embedding quality = RAG quality. Most bad RAGs are retrieval failures, not generation failures.

10. Conclusion

RAG turns a general LLM into an assistant that answers from **our own, current documents**: fewer invented answers, sources the user can check, and new knowledge added by simply adding documents. It has two phases: indexing (load, chunk, embed, store) and querying (embed, search, retrieve, augment, generate). Chunking and embeddings decide what the LLM gets to see, so they decide the quality of the answer. Start simple with recursive chunking, use the same embedding model everywhere, and measure retrieval on real questions before tuning anything else.